5. Creating Lambda Functions Using the AWS SDK for Python - Simulation in Docker

Go to **https://killercoda.com/docker/scenario/container-build** go to terminal and perform the following tasks

To create a Lambda function in in killercoda docker lab environment, you'll likely want to simulate AWS Lambda locally, as Play with Docker provides Docker environments, **not** direct access to AWS services. Here's how you can simulate AWS Lambda locally using Docker and tools like **AWS Lambda Runtime Interface Emulator** or **LocalStack**.

Solution:: Use AWS Lambda Runtime Interface Emulator (RIE)

You can run your Lambda function inside a Docker container using AWS's base images and the RIE.

1. Create a Lambda Function (Python example)

Create a file called `lambda_function.py`

(vi lambda_function.py)

```
def lambda_handler(event, context):
    return {
        "statusCode": 200,
        "body": "Hello from Lambda!"
    }
```

2. Create a Dockerfile (vi Dockerfile)

```
FROM public.ecr.aws/lambda/python:3.9

# Copy function code

COPY lambda_function.py .

# Set the CMD to your handler

CMD ["lambda_function.lambda_handler"]
```

3. You may find the internal IP of node1 (currently running terminal) and note it down:

```
hostname -I
```

You get the IP address. (If you get 3, select the first one; That is the Ip address of the node)

For Example, `192.168.x.x` is your Node 1's internal IP. (Note it down and use at last step from terminal 2)

4. Build the Docker Image

docker build -t my-lambda .

5**. Run the Container Locally**

docker run -p 9000:8080 my-lambda

This starts the Lambda runtime in the container and listens on port `9000`.

Now Click on Add New Instance; a new node will be created and a corresponding new terminal will be opened. Then go to the next step.

6. Invoke the Function

In another terminal in Play with Docker, use `curl` as follows: (type the command statement in single line i. curl -XPOST “http://….)

curl -XPOST "http://localhost:9000/2015-03-31/functions/function/invocations" -d '{}'

You may replace that localhost with the internal ip of node 1 which you have noted down earlier. This establishes the connection among those two terminals / establishes the link with a common container.

For Example: If the IP of node 1 is inet 192.168.0.15/24 then the above command will be

curl -XPOST "http://192.168.0.15:9000/2015-03-31/functions/function/invocations" -d '{}'

You should get a response like in Terminal 2

```
{
  "statusCode": 200,
  "body": "Hello from Lambda!"
}
```

And a *success* message at terminal 1.